Krotki, słowniki, zbiory

Lab 8

Krotka - definicja

- Jest niezmienna (immutable)
  – po utworzeniu krotki nie można modyfikować
- Krotke tworzą elementy oddzielone przecinkiem i otoczone nawiasami ()

krotka = (1, ”Ptyhon”, 3.14)

- Krotka może zawierać różne typy danych.
- Dostęp do elementów jest tak jak dla listy.

Właściwości krotek (,)

- Dostęp do elementów przez indeksy

```
2   krotka = (1, "Python", 3.14)
3   print(krotka[1])  # Python
```

- Niezmienność → **brak możliwości modyfikacji elementów**
- Mogą być również elementami słowników lub list

```
5   lista = [(1,"ala"), ("ola", ("tola",3.14) )]
6   print(lista[1][1][0])  # tola
```

Zastosowania krotek

- Grupowanie danych w logiczne całości.
- Zwracanie wielu wartości z funkcji.

```
 8   def podziel(a, b):
 9       return (a // b, a % b)
10
11   wynik = podziel(10, 3)  # (3, 1)
12   print(wynik[0])  # 3
13   print(wynik[1])  # 1
```

- Klucze w słownikach (bo są niezmienne).

Słownik: definicja

- Struktura danych typu: **{ klucz : wartość , klucz : wartość }**
- Tworzony za pomocą: **{ }**
- **Klucze muszą być niezmienne** (np. string, liczba, krotka)

```
15    slownik = {"imie": "Anna", "wiek": 25}
16
17    print(slownik["imie"])  # Anna
18    print(slownik["wiek"])  # 25
```

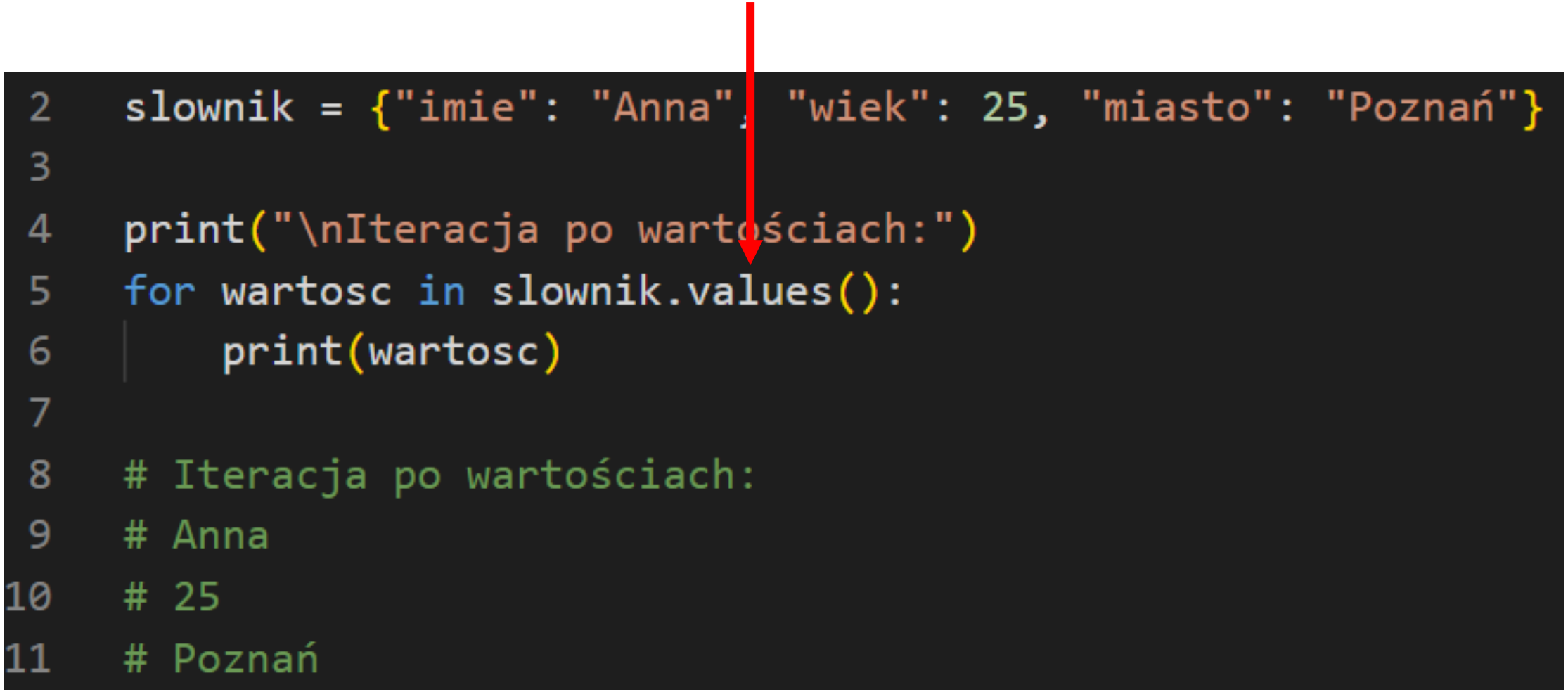
Operacje na słownikach

- Dodawanie i modyfikacja wartości
- Usuwanie elementów (del, .pop())
- Iteracja po: parach, wartościach, kluczach

```
20  slownik = {}
21  slownik["miasto"] = "Poznań"
22  slownik["państwo"] = "Polska"
23  slownik[10] = ("lista", [1,2,3])
24  for k, v in slownik.items():
25      print(k, v)
26  print("----")
27  del(slownik ["państwo"]);
28  for k, v in slownik.items():
29      print(k, v)
```

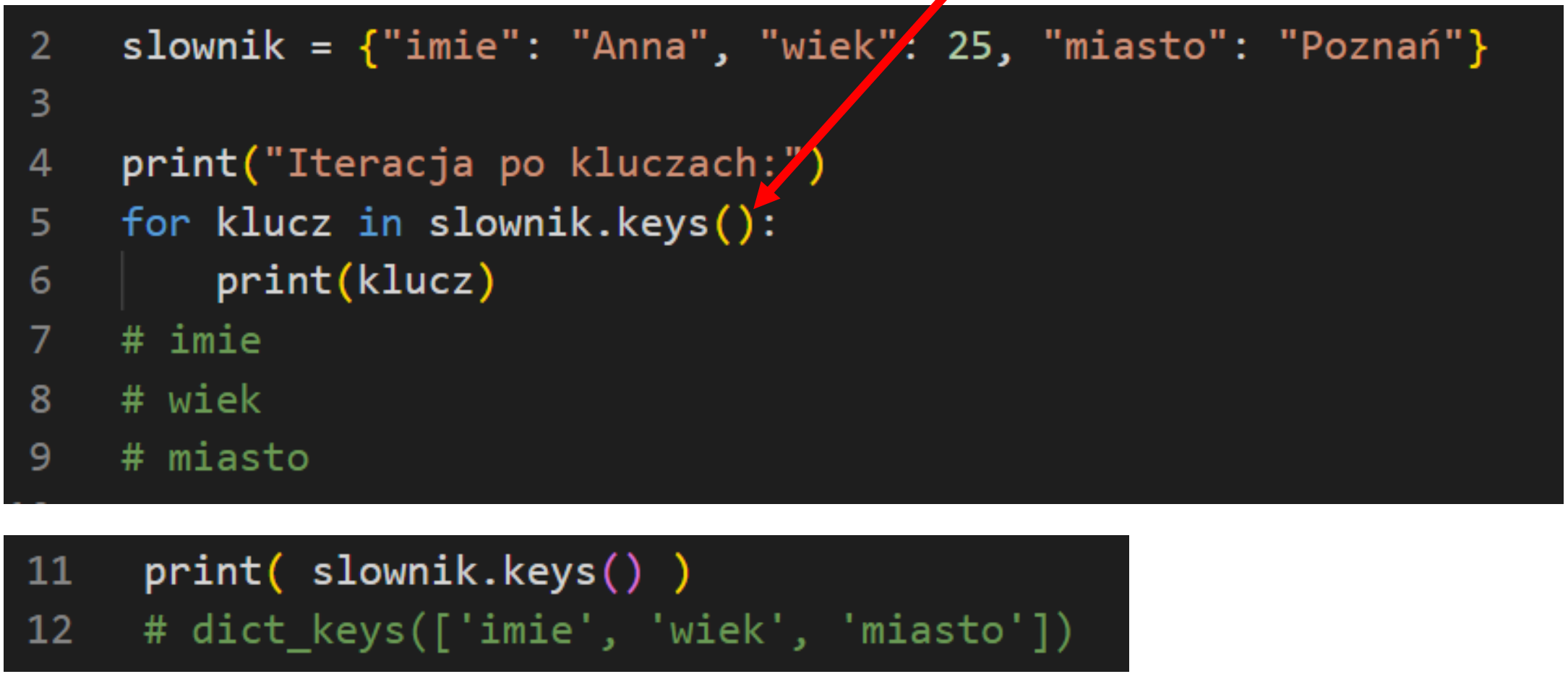
```
31  # miasto Poznań
32  # państwo Polska
33  # 10 ('lista', [1, 2, 3])
34  # ----
35  # miasto Poznań
36  # 10 ('lista', [1, 2, 3])
```

Słownik – iteracja po wartościach



```
2   slownik = {"imie": "Anna", "wiek": 25, "miasto": "Poznań"}
3
4   print("\nIteracja po wartościach:")
5   for wartosc in slownik.values():
6       print(wartosc)
7
8   # Iteracja po wartościach:
9   # Anna
10  # 25
11  # Poznań
```

Słownik – iteracja po kluczach



```
2   slownik = {"imie": "Anna", "wiek": 25, "miasto": "Poznań"}
3
4   print("Iteracja po kluczach:")
5   for klucz in slownik.keys():
6       print(klucz)
7   # imie
8   # wiek
9   # miasto
```

```
11   print( slownik.keys() )
12   # dict_keys(['imie', 'wiek', 'miasto'])
```

•.items() → zwraca pary (klucz, wartość)

Słownik – wybrane metody

- **.keys()** → zwraca wszystkie klucze
- **.values()** → zwraca wszystkie wartości
- **.items()** → zwraca pary (klucz, wartość)
- **.get(klucz,[domyślna])** → pobiera wartość dla klucza, jeśli nie istnieje zwraca domyślna
- **.update(inny_slownik)** – aktualizuje słownik o nowe pary klucz–wartość
- **.pop(klucz, [domyślna])** – usuwa element o podanym kluczu i zwraca jego wartość
- **.popitem()** – usuwa i zwraca ostatnią parę (klucz, wartość)
- **.clear()** – usuwa wszystkie elementy ze słownika

Zbiory – definicja

- Zbiór = kolekcja **unikalnych** elementów
- Zbiór jest tworzony za pomocą **{}** lub **set()**

```
2   A = {1,2,12,4,5}
3   B = set([1,2,12,4,5])
4   print(A)
5   print(B)
6   print(type(A))
7   print(type(B))
8   C = set("Ala ma kota")
9   print(C)
10  print(len(C))
11  D = set()
12  print(len(D))
13  E = {}
14  print(type(E))
```



```
{1, 2, 4, 5, 12}
{1, 2, 4, 5, 12}
<class 'set'>
<class 'set'>
{'l', 'o', 'A', 'a', ' ', 'm', 'k', 't'}
8
0
<class 'dict'>
```

Zbiory – właściwości

- Elementy są **unikalne**
- Brak gwarancji kolejności
- Zmienna struktura (mutable)

Zbiory – metody

- **.add(element)** – dodaje nowy element do zbioru
- **.remove(element)** – usuwa element; zgłasza błąd jeśli go nie ma
- **.discard(element)** – usuwa element; nie zgłasza błędu jeśli brak
- **.union(inny_zbior)** – zwraca sumę zbiorów (wszystkie elementy)
- **.intersection(inny_zbior)** – zwraca część wspólną zbiorów
- **.difference(inny_zbior)** – różnica zbiorów
- **.issubset(inny_zbior)** – sprawdza, czy zbiór jest podzbiorem
- **.issuperset(inny_zbior)** – sprawdza, czy zbiór jest nadzbiorem
- **.isdisjoint(inny_zbior)** – sprawdza, czy zbiory są rozłączne